

Machine and Representation Learning Based GNSS Spoofing Detectors Utilizing Feature Set From Generic GNSS Receivers

Asif Iqbal, *Member, IEEE*, Muhammad Naveed Aman, *Senior Member, IEEE*,
and Biplab Sikdar, *Senior Member, IEEE*

Abstract—The Global Navigation Satellite System (GNSS) plays critical role in providing precise timing and positioning information for various applications. However, its civilian signals are vulnerable to spoofing attacks, necessitating robust detection methods. While supervised learning has shown promise in spoof detection, it requires distinct features in the training data for effective learning. In this study, we propose a novel training feature set that combines power and Signal Quality Monitoring (SQM) metrics from a single-antenna GNSS receiver. We introduce a Two-Stage Artificial Neural Network (TS-ANN) that leverages this feature set alongside multi-correlator finger values for effective spoof detection. Furthermore, supervised learning models often struggle with previously unseen attacks due to limited overlap with the training data. To address this, we present a zero-day attack detector based on unsupervised representation learning using a Variational Autoencoder (VAE) which is trained exclusively on authentic datasets, enhancing its ability to detect novel attack patterns. Experimental results on publicly available TEXBAT datasets demonstrate that our TS-ANN achieves a detection probability (PD) exceeding 99% for similar test datasets. In more sophisticated attack scenarios, such as DS-7, performance may decrease to 50.68%. Nevertheless, our zero-day detector maintains a PD exceeding 92.5%, highlighting its resilience to previously unseen attacks.

Index Terms—GNSS attack variation, GNSS spoofing detection, neural networks, signal quality monitoring (SQM), TEXBAT, variational autoencoder, zero-day detection.

I. INTRODUCTION

GLOBAL navigation satellite system (GNSS) receivers have received massive popularity in applications where timing or navigation information is central for their operations. Due to the open-access nature of civil GNSS signal standards, developing receivers for these signals has become straightforward, contributing to their wide availability and low cost [1]. As a by-product of the open-access nature of civilian GNSS signals, these signals are fully predictable. Therefore, a malicious user can carry out spoofing attacks against GNSS receivers and fool them to report misleading time or position information [2]. Thus, these spoofing attacks are considered a serious threat to the infrastructure and applications that are reliant on GNSS.

This work was supported in part by Ministry of Education, Singapore under grants A-0009040-00-00 and A-0009040-01-00.

A. Iqbal, and B. Sikdar are with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore 117583, e-mail: {aiqbal, bsikdar}@nus.edu.sg

M. N. Aman is with the School of Computing, University of Nebraska-Lincoln, Nebraska, USA, e-mail:naveed.aman@unl.edu

With increasing accessibility to programmable simulators and software-defined radios, executing effective GNSS signal spoofers has become easier. These spoofing attacks can be categorized based on their complexities: simplistic, intermediate, and sophisticated attacks [3]. Simplistic attacks involve generating high-powered spoofed signals near the target receiver and are relatively easier to detect. On the other hand, sophisticated attacks can exploit multiple transmit antennas that can effectively bypass the direction of arrival anti-spoofing measures. Most importantly they require precise knowledge of the receiver antenna position and a precise fading model of spoofer to receiver antenna signal path to be able to perform an accurate carrier-phase alignment with the authentic signals, making them more difficult to execute [3], [4]. However, in scenarios where the attacker has physical access to the receiver antenna, as is often the case in applications like shipment vehicle monitoring, such an attack can be executed. In contrast, intermediate-level spoofing attacks maintain whatever initial phase offset arises between signals throughout the attack [5]. Induced spoofing or tracking-point-carry-off spoofing attack strategies, are easier to execute at intermediate-level than sophisticated ones and are more effective than simplistic ones. Under such attacks, the spoofer initially aligns the carrier and code phase of the spoofed signal with the authentic signal at the target receiver's correlator tracking loop, and maintains whatever initial phase offset arises between signals throughout the attack [5]. Subsequently, the attacker gradually takes over the correlation peak by altering the power and code rate of the spoofed signal [3]. Without countermeasures, the target receiver may fail to detect the spoofing process, allowing the spoofer to manipulate the GPS signals' timing and positional information, potentially leading to significant harm.

A vast array of spoofing detection techniques have been presented in the literature, utilizing GNSS signal and receiver properties. These techniques can be broadly classified into authentication-based, multiple-antenna-based, inertial- and sensor-based, and single-antenna-based methods [3]. Single-antenna-based techniques, which are practical and easily deployable, stand out among these methods. They are low-cost, require no additional hardware, and can be implemented through firmware updates. These techniques include monitoring the received power directly [6] or through an automatic gain control (AGC) unit [7]; and techniques that monitor the auto-correlation profile of a receivers' tracking loop through signal quality monitoring (SQM) [4], [8]–[10]; and those that

are a combination of both [3], [11]. Most of these methods use the Bayesian detection framework to perform spoof detection.

To be concise, Bayesian frameworks require careful selection of the signal model, prior and/or likelihood functions, and rely on changes in feature distribution in the presence or absence of a spoofed signal. Only then, can an appropriate threshold be confidently selected for a given detection probability. In contrast, machine learning (ML) based methods, including artificial neural networks (ANN) have been shown to work well for solving complex problems that are difficult to solve using traditional modelling techniques. These methods do not require careful specifications of mathematical models, noise models, or their statistics. For the same reason, ANNs are particularly well-suited for addressing the challenging problem of spoofing detection.

Recently a number of ML based methods have also been proposed for GNSS spoofing detection. These methods use features extracted from acquisition, tracking, and position-velocity-time (PVT) blocks of a typical GNSS receiver as inputs to train various supervised ML models for classification. Bose *et al.* [12] trained an ANN with input features C/N_0 , pseudorange, carrier phase, and doppler shift for spoof detection. Aissou *et al.* [13] used 13 features from tracking and PVT solution blocks to train multiple ML based classifiers. Shafiee *et al.* [14] used ANN, Naive Bayes, and K-NN to train classifiers using the received power and two SQM (delta and early-late phase) metrics. Similarly, Manesh *et al.* [15] used pseudorange, doppler shift, and signal-to-noise (SNR) as inputs to an ANN based classifier. In [16], Borhani *et al.* used the Cross Ambiguity Function (CAF) generated at the receivers' acquisition block for spoof detection by detecting the existence of multiple peaks in the CAF using an ANN and a Convolutional NN (CNN). Similarly, in [17], Li *et al.* used a Generative Adversarial Network (GAN) to detect spoof signal detection in CAF. Both of these methods are computationally expensive as they require regular CAF computations at the acquisition block to ensure robust detection. In another recent work, Kim *et al.* [18] used the location data extracted from the PVT block to generate differential features that are used to flag irregularities in the resulting mobility profile of the receiver. They trained multiple ML classifiers and reported an impressive 99% detection accuracy on VeReMi dataset.

The use of supervised ML-based methods has demonstrated promising results, making them a valuable defense mechanism against GPS spoofing attacks. One of the main difficulties with supervised learning is that models often have poor performance when tested on datasets that are substantially different (or unseen) from the ones used for training. Another key shortcoming of the above mentioned ML based methods is their lack of testing on a wider range of spoofing scenarios. Almost all of them use datasets generated in-house instead of using standard spoofing scenarios like the ones provided in the TEXBAT dataset [5] or other similar datasets. Thus, it will be interesting to investigate ML based methods on such standard datasets as well.

Detecting GNSS spoofing attacks can be challenging due to the different levels of power advantage that spoofed signals might have over the authentic signals. In the case of high

power advantage, SQM metrics remain relatively unchanged, but the increase in received power or carrier to noise spectral density ratio (C/N_0) can easily flag the presence of a spoofed signal at the receiver. Whereas, in case of a low power advantage, either the attack will be ineffective, or will result in serious auto-correlation function distortion due to interactions between similar powered authentic and spoofed signals. In this case, SQM metrics can be very effective in detecting such attacks during initial carry-off of the correlation peak [3], [11]. However, these metrics are only useful as transient detectors and are not effective in detecting spoofed signals once they have moved the peak by approximately one chip duration. Similarly, received power and C/N_0 can be considered as static indicators. Assuming that the spoofer is not able to completely block or nullify the authentic GNSS signal, methods that combine abnormal power detection and SQM metrics into a single detection scheme can serve as a robust defense against spoofing attacks [3], [11].

In recent years, a number of SQM metrics have been proposed, from classical three (early-prompt-late) correlator tap SQM metrics like ratio, delta [19], early-late-phase [20], symmetric differences [3]; to multi-tap SQM metrics like Manfredini [11] and WSCM [9]. In [4], Sun *et al.* used three SQM metrics to create a composite one which was able to perform better (in terms of detection probability) when compared with its constituents. This performance gain can be attributed to different SQM metrics being sensitive to different time points during the peak carry-off.

Motivated by these observations, in this paper, our input features vector for each time point consists of received power, C/N_0 , and 5 different SQM metrics. As shown in Fig. 1, the received power is computed at the output of radio frequency (RF) block and the remaining features are readily available (or can be computed) at the tracking block of a typical off-the-shelf GNSS receiver, making it practical and cost effective. This feature set enables the detection of presence of a spoof signal even before (after) the correlation peak carry-off begins (completes). Our focus is to analyze ML classifiers in terms of their detection performance and resilience against different spoofing attacks. Baselines for this performance evaluation are four supervised ML and an ANN model. In addition to these, we propose a (supervised learning based) two-stage ANN (TS-ANN) model which at its first stage generates a new SQM metric using the multiple correlator fingers as input in a data-driven way to maximize detection performance. Supervised learning relies on having a representative training dataset for accurate performance on test data. However, models tested on significantly different datasets can result in low accuracy.

To address this challenge, unsupervised representation learning-based methods identify one class from the training dataset and classify test samples accordingly. This approach improves accuracy on new and unseen datasets. To achieve robust spoof detection, we propose a variational autoencoder (VAE)-based zero-day attack detector that learns an underlying latent distribution for authentic data, which is then used to identify unseen attack patterns. The base learners in both proposed methods are the fully connected ANNs, however, both models undergo distinct training approaches:

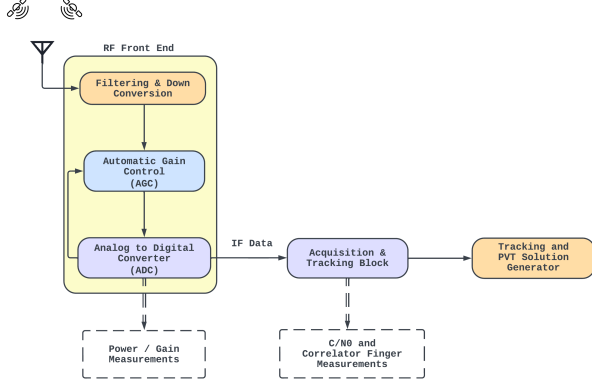


Fig. 1. Block diagram of a typical single antenna GNSS receiver. Solid arrows depict normal data flow and dashed arrows show feature extraction.

TS-ANN employs supervised learning as a binary classifier, while the VAE employs unsupervised learning to minimize reconstruction errors and align the latent distribution with a centered Gaussian distribution. Results show that this zero-day detector (ZDD) outperforms supervised learning methods on unseen data. To summarize, we make the following major contributions in this paper:

- Propose a novel feature set that combines features extracted at RF and tracking blocks of a GNSS receiver.
- Develop a TS-ANN that uses multiple correlation finger values to generate a new SQM metric during the initial stage which is subsequently integrated with the remaining features and utilized as inputs for the second ANN stage.
- Propose a zero-day attack detector based on a VAE that is resistant to attack variance. Our experiments demonstrate that this detector outperforms other supervised ML-based methods for sophisticated attack scenario.
- Evaluate our methods on multiple attack scenarios, including static and mobile GNSS receivers, and simple to sophisticated spoofing scenarios from the public TEX-BAT data recordings.

II. METHODOLOGY

A. Tracking Phase Signal Model

Assuming a spoof-free GPS scenario, the authentic complex-valued L1-C/A signal at the output of the receiver's RF chain can be expressed as follows:

$$r_A(t) = \sqrt{P_A} D(t - \tau_A) C(t - \tau_A) \exp(j\theta_A), \quad (1)$$

where, P_A represents the received authentic signal power, t is the time in seconds, $D(t)$ is the BPSK-modulated navigation data, $C(t)$ is the BPSK-modulated pseudorandom spreading code, τ_A represents code phase, and θ_A represents carrier phase in radians. It is worth noting that the values of P_A , τ_A , and θ_A are all time-varying, but for simplicity, their notation has been suppressed here. We assume $D(t) = 1$ without loss of generality and ignore it in the subsequent analysis [3].

In the presence of a spoofed signal at the input of a GPS receiver, the output of the tracking correlators is given by [3]:

$$\begin{aligned} I_d &= \sqrt{P_A} R(dT_c) \cos(\theta_A) + \sqrt{\gamma P_A} R(dT_c - \Delta\tau) \cos(\theta_A + \Delta\phi) + \xi_d^I, \\ Q_d &= \sqrt{P_A} R(dT_c) \sin(\theta_A) + \sqrt{\gamma P_A} R(dT_c - \Delta\tau) \sin(\theta_A + \Delta\phi) + \xi_d^I, \\ \Psi_d &= I_d + Q_d i, \end{aligned} \quad (2)$$

where I_d and Q_d represent the in-phase and quadrature components of the correlators' output. T_c is the chip duration and, along with a unitless d (dT_c), determines the spacing for prompt ($d = 0$), early ($d < 0$), and late ($d > 0$) correlators. ξ_d^I and ξ_d^Q are the zero-mean Gaussian thermal noise with a constant noise spectral density N_0 present in the I and Q channels, respectively. $\gamma = P_S/P_A$ denotes the average power advantage of the spoofing signals over the authentic one over the integration period. $\Delta\tau$ and $\Delta\phi$ represent the delay and phase difference of the spoofed signal relative to the authentic signal. Ψ_d represents the complex-valued correlator output. The normalized ideal autocorrelation function of a BPSK-modulated signal is given by $R(\cdot)$ and is represented as:

$$R(dT_c) = \begin{cases} 1 - |dT_c|/T_c, & |dT_c| \leq T_c, \\ 0, & |dT_c| > T_c. \end{cases} \quad (3)$$

B. Feature Extraction

We chose to use only runtime features from any GPS receiver for practical and easily deployable GPS spoofing detection. Unlike features used by others in [12], [13] that require additional processing time (features coming from PVT module), our approach using only 7 real-time features obtained from the RF and tracking blocks of a generic GPS receiver creates a more efficient and versatile solution. Details of the extraction methodology are provided below.

1) *Received Power*: To account for the fact that the majority of signal power in the L1 GPS band is concentrated in a 2 MHz band around the L1 carrier frequency, a 2 MHz bandwidth filter is used to filter the RF output signal. If $\tilde{r}(t)$ denotes the filtered version of the RF output $r(t)$, the received power P_k (in dBW) is then determined by computing the average power over an interval from t_{k-1} to t_k :

$$P_k \triangleq 10 \log_{10} \left(\frac{1}{T} \int_{t_{k-1}}^{t_k} |\tilde{r}_C(t)|^2 dt \right). \quad (4)$$

2) *Carrier to Noise Ratio*: The strength of received GPS signals is often measured by the C/N_0 metric, although it cannot be directly measured and requires estimation. One common method for estimating C/N_0 is the Narrow-band Wide-band Power Ratio (NWPR) method [21, 1.7.3]. To aid in differentiating between genuine interference and spoofing scenarios, as discussed in [11], both the received power and C/N_0 are considered in the trained classifier.

3) *Signal Quality Monitoring*: In order to capture the auto-correlation profile distortion during the spoofed and authentic signal interaction, we use 5 different SQM metrics as input features. These metrics are given below:

- Ratio Metric [19]

$$m_{ratio} \triangleq \frac{I_{-d} + I_{+d}}{I_P} \quad (5)$$

- Delta Metric [19]

$$m_{delta} \triangleq \frac{I_{-d} - I_{+d}}{I_P} \quad (6)$$

- Early Late Phase Metric [20]

$$m_{elp} \triangleq \tan^{-1} \left(\frac{Q_{-d}}{I_{-d}} \right) - \tan^{-1} \left(\frac{Q_{+d}}{I_{+d}} \right) \quad (7)$$

- Symmetric Differences [3]

$$m_{sd} \triangleq \frac{|\Psi_{-d} - \Psi_{+d}|}{\sigma_{N_0}} \quad (8)$$

- Manfredini Metric [11]

$$m_{fred} \triangleq \frac{|L_x - E_x|}{|\Psi_P|}. \quad (9)$$

Here the variables $I_{\pm d}$, $Q_{\pm d}$, and $\Psi_{\pm d}$ are given in (2) with $d = 0.5$. I_P and Ψ_P are the respective prompt correlator values. m_{fred} is computed using 9 correlators evenly spaced between $d = [-0.1016, 0.1016]$, with L_x and E_x being the linear combination of complex values from late and early correlator fingers, respectively. Finally, σ_{N_0} is the standard deviation of Ψ_{-2} during the spoof free case.

The reason behind selecting five distinct SQM metrics is their complementarity. This means that they collectively provide a more comprehensive assessment of the receiver's performance. As noted in [4], when the m_{elp} value is high, the values of m_{delta} and m_{ratio} tend to decrease, and vice versa. Another metric, m_{sd} , considers the absolute difference between early and late correlators, scaled by the standard deviation of the noise (in the absence of spoofing). It exhibits high values during and after complete takeover of the receiver's correlator peak. Finally, m_{fred} is sensitive to slight distortions that occur at the beginning of code pull-off, as it employs correlator fingers that are located very close to the prompt.

C. Machine Learning Classifiers

In this study, we use four highly used ML classification models and a fully connected ANN as the baselines for our performance evaluation. The chosen ML models include random forest classifier (RFC), extreme gradient boosting (XGB), K-nearest neighbours K-NN, and support vector machines (SVM). For details, reader is referred to [22]. We utilized the grid search method to fine-tune the hyperparameters of our ML algorithms for optimal performance. The RFC, XGB, K-NN, and SVM models were all implemented using the Scikit-Learn [23] Python library. For RFC and XGB models, we set the number of estimators to 8 and 20, respectively. The K-NN model used 8 neighbors, while the SVM employed the radial basis function *RBF* kernel. All other hyperparameters were left at their default values.

The ANN was implemented using the Pytorch [24] Python library, and we conducted hyperparameter tuning to optimize its performance. Specifically, we experimented with the number of layers and nodes, activation functions, and learning rates. Our final model consisted of 3 layers with 20 – 10 – 1 nodes, respectively. We used PReLU activation functions for the first two layers and a Sigmoid function for the final layer.

III. PROPOSED GNSS SPOOFING DETECTION MODELS

In this section, we discuss the two GNSS spoof detection methodologies and their respective detection protocols.

A. Two-Stage Artificial Neural Network

In a standard ANN, data features are processed through successive layers of nodes to produce an output. Each node in a layer combines inputs with associated weights, followed by a non-linear activation function. More nodes and layers enable the learning of complex functions. For GNSS spoof detection, a binary classification task, the ANN's output is binary and the entire model is trained using the back-propagation algorithm.

The success of an ANN depends heavily on the features used during its training. The model is easier to train and its performance is better if more informative and better features are used. With this in mind, there is a desire to augment the established feature set with additional information. To achieve this, the proposed approach involves using five pairs of correlators positioned on either side of the prompt correlator at $\pm 1.0, \pm 0.75, \pm 0.5, \pm 0.25, \pm 0.1$. Drawing inspiration from the concept presented in [9], which suggests that optimal linear coefficients applied to multiple correlator finger values can enhance the SQM metric and thereby improve spoof detection, our work adopts a similar approach. Here, we combine the mentioned correlator fingers with the prompt correlator values to create a novel SQM metric. This new metric can be combined with the original feature set to perform classification. The goal is to learn this new metric in a supervised data-driven manner so that the model can encode any contrastive information that may be otherwise missing from the original feature set.

Fig. 2 illustrates the proposed Two-Stage ANN (TS-ANN) architecture. Here the original feature set is denoted as a vector x_1 , while the multi-correlator finger values, which were introduced previously, are represented by x_2 . The target label is denoted by y . The architecture includes two conjoined neural networks, namely $f_1(\cdot)$ and $f_2(\cdot)$ as shown in Fig. 2. The final activation function of $f_2(\cdot)$ is the *Sigmoid* σ function, which outputs values in the interval $(0, 1)$. On the other hand, the final activation function of $f_1(\cdot)$ is the *Tanh* function, which outputs values in the interval $(-1, 1)$. The choice of *Tanh* function is motivated by the need to keep the learned SQM metric between -1 and 1, which facilitates its visualization. The parameteric ReLU activation function was used for all other inner nodes of both networks. The TS-ANN network architecture was finalized after careful selection of number of layers, number of nodes in each layer, and activation functions for these nodes. In addition, objective was to keep the parameters to a minimum to keep the computational

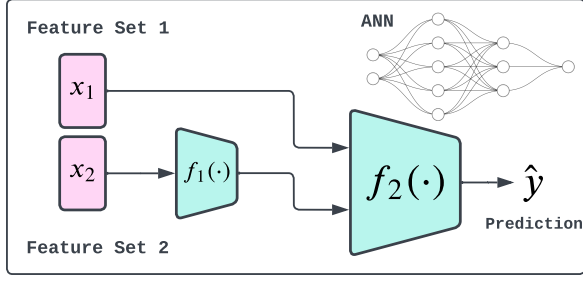


Fig. 2. Block diagram of the proposed Two-Stage ANN model. $f_1(\cdot)$ and $f_2(\cdot)$ are two separate ANN models.

complexity low. The best structure that was small enough while giving the best overall performance was $20 - 10 - 1$ nodes for $f_2(\cdot)$ and $5 - 1$ nodes for $f_1(\cdot)$. Adding more layers did result in slight improvements, but those improvements were not justifiable to the additional computational burden associated with them. The overall network can be summarized as $\hat{y} = f_2([x_1, f_1(x_2)])$.

B. Zero-day Detection Framework

Classical supervised model training relies on having a diverse set of training data that encompasses all possible data classes, including both normal and spoofed data samples. This approach is highly effective when the training and test datasets share a similar distribution. However, it faces significant challenges when encountering novel attack scenarios that deviate markedly from the learned distributions. In such cases, the trained classifiers may struggle to detect these unfamiliar threats. On the other hand, zero-day detection, or the ability to identify previously unseen attack instances that differ from known representations, represents a critical capability in cybersecurity [25]. Traditional supervised learning models often fail in zero-day scenarios due to their dependence on predefined attack patterns from the training data. Unsupervised learning techniques, such as VAE [26], GAN [27], and Deep Belief Networks (DBN) [28], offer promising solutions for zero-day detection. These models rely on profiling based techniques, where they are trained solely on normal data samples. Once trained, they evaluate whether a test sample conforms to the learned distribution of normal data. If the test sample deviates significantly from this distribution, it is flagged as potentially anomalous, making these models valuable tools for identifying previously unseen attack instances.

This work leverages the capabilities of a VAE, a deep learning model, for zero-day detection. By training the VAE on a substantial corpus of genuine GPS data, the model learns the underlying latent patterns and features that define genuine GPS samples. Consequently, the VAE can effectively distinguish between test samples that align with genuine patterns and those that exhibit deviations, enabling zero-day detection of input samples that have not been previously encountered.

1) *Variational AutoEncoder*: There are various types of autoencoder (AE) models in the literature [29]. One of the most popular variants is the VAE [26], shown in Fig. 3. Unlike the vanilla AE, the VAE employs a probabilistic approach to

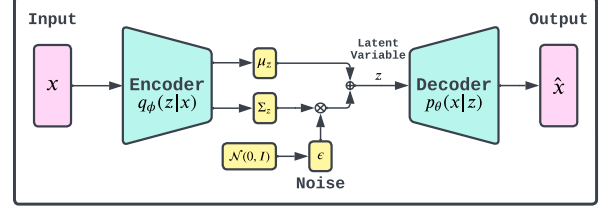


Fig. 3. The Variational Auto-Encoder architecture.

the latent vector z . Specifically, the VAE encoder outputs two vectors, a mean vector μ_z and a variance vector Σ_z , which parameterize a Gaussian distribution $q_\phi(z|x)$ from which the latent vector z is sampled as $z = \mu_z + \epsilon \Sigma_z$, where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the noise vector. This formulation ensures that for a given input x , different sampled latent vectors are generated, leading to a more diverse latent space. Consequently, the VAE decoder learns to map a region of the latent space to the output, instead of relying on a single point. This property makes the VAE decoder suitable for generating samples that are similar but not identical to the training samples. Given an input x , a probabilistic encoder maps it to the latent vector z according to the distribution $q_\phi(z|x)$, and a probabilistic decoder reconstructs the output x from z using the distribution $p_\theta(x|z)$. The VAEs are trained by maximizing the following loss function

$$\mathcal{L}(\theta, \phi; x, z) = \mathbb{E}_{z \sim q_\phi(z|x)} (\log p_\theta(x|z)) - D_{KL}(q_\phi(z|x) || p(z)). \quad (10)$$

Equation (10) is also called the variational lower bound. The first term in (10) is the reconstruction loss computed using the mean square error between the input and reconstructed samples. The second term is the Latent loss which uses Kullback-Leibler divergence (KLD) to minimize the distance between the learned latent distribution $q_\phi(z|x)$ and a prior distribution $p(z)$, typically selected as standard Normal $\mathcal{N}(0, 1)$. The KLD from q to p distribution is given by

$$D_{KL}(q||p) = \sum_x q(x) \log \frac{q(x)}{p(x)}. \quad (11)$$

The Latent loss in (10) takes a high value when the latent probability distribution is further away from a standard Normal distribution, and takes a small value when both the distributions are close by. This KLD term solves a couple of issues with the vanilla AEs, i.e., the latent space is forced to be centered at the origin and to be symmetric around it ensuring that there are no gaps in the latent space. Moreover, the latent distribution can be made to match any arbitrary distribution $p(z)$ as the choice is not limited to standard Normal.

2) *Zero-day Detector (ZDD)*: In this paper, we adopt a VAE as our baseline model to learn an efficient representation of only genuine data samples. For a given genuine clean data sample x , the VAE's encoder maps it into μ_z and Σ_z vectors generated by the latent distribution $q_\phi(z|x)$. The latent variable z is sampled from a Gaussian distribution parameterized by μ_z and Σ_z as $z = \mu_z + \epsilon \Sigma_z$, where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the noise vector. This mapped variable z is then fed into the

VAE's decoder, which samples the reconstructed output $\hat{x}$ from $p_\theta(x|z)$. Since the input sample is taken from the genuine dataset, the reconstruction error $\eta(x) = \|x - \hat{x}\|_2$ will be small and bounded. Here, $\|\cdot\|_2$ denotes the ℓ_2 vector norm. On the other hand, for a spoofed data sample, the reconstruction error will be high since the VAE has not seen such samples while training. Therefore, we can use the reconstruction error $\eta(x)$ as an indicator function, where if $\eta(x) \leq \tau$, then x is classified as a genuine sample, and spoofed otherwise.

We designed a VAE encoder comprised of a 3-layer NN with 20–10–4 nodes, with PReLU activation functions used for all but the final layer, which has the latent dimension set to 2. The VAE decoder consisted of a 3-layer NN with 10–20– n nodes, where n represents the input dimensions. PReLU activation functions were used for all inner nodes, while the final layer had no activation function. The selection of an appropriate decision threshold τ is critical for the optimal performance of the proposed ZDD. To ensure the robustness of τ , we link it to the desired input false positive rate (FPR) or probability of false alarm (PFA) when testing on the training dataset. This process involves the following steps:

- 1) Compute the reconstruction errors of the entire training dataset.
- 2) Calculate the empirical cumulative distribution function (CDF) based on these errors.
- 3) Determine the error value above which the CDF remains strictly below $(1 - \text{FPR})$.
- 4) Save this computed value as the threshold τ for classifying the test samples.

This approach ensures that the threshold τ is effectively tailored to meet the desired false positive rate during testing. For instance, if we set FPR at 5%, then the samples lying above the 95th percentile of the reconstruction errors' CDF will be classified as anomalous or spoofed. In the experimental section, we evaluated the model's performance under various FPR settings, namely, [5%, 1%, 0.1%] which is a typical design choice for a detector.

IV. EXPERIMENTAL EVALUATION

A. Dataset Description

The TEXTBAT dataset is a collection of publicly available high fidelity binary recordings that depict different spoofing scenarios performed on civil GPS L1 C/A signals [5]. The dataset is made available by the Radio Navigation Laboratory (RNL) of the University of Texas in Austin. The recordings are centered at a carrier frequency of 1575.42 MHz and have a bandwidth of 20 MHz. The samples are 16 bits with a complex sampling rate of 25 Msps. TEXTBAT is one of a few publicly available datasets for testing anti-spoofing GPS receivers. An attribute summary of the various spoofing attack scenarios is given in Table I.

In Table I, the term “Code Phase Prop.” indicates that the carrier phase of the spoofed signal changes proportionally to the code phase. “Frequency Lock Mode” refers to the scenario in which the initial phase offset between the spoofed and the authentic signal remains constant throughout the spoofing scenario. The term “Carrier Phase Aligned” suggests that

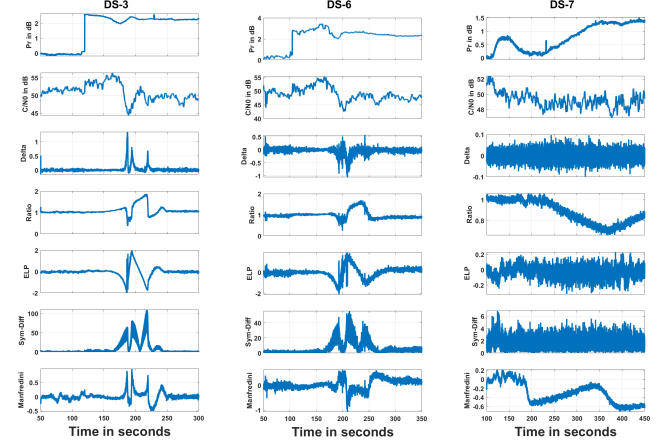


Fig. 4. DS-3 (PRN-23), DS-6 (PRN-22), and DS-7 (PRN-23) features.

the spoofed signal aligns precisely with the carrier phase of the authentic signal. “Matched” implies that the power of the spoofed signals is matched, but the exact values are not specified. “Onset” is the time at which the spoofed signal is first observed at the receivers’ antenna. Lastly, “Spoofing Type” refers to time or position information deviation induced by the spoofer. The clean-dynamic dataset was recorded using a car mounted antenna traveling in the city of Austin, Texas, and clean-static was recorded with an antenna placed on top of the RNL lab. In this study, we utilize all datasets given in Table I, including both static and dynamic scenarios, where DS-2 and DS-5 are the simplistic attacks such that the spoofer has significant power advantage over the clean signal. DS-3, 4, and 6 are scenarios with intermediate difficulty as their relative power advantage is small and the spoofer has engaged a frequency lock onto the clean signal. Finally, DS-7 is the most challenging scenario as it not only has relatively matched power, but it also performs carrier phase alignment.

B. Feature Extraction and Model Training

The TEXTBAT datasets were analyzed using FGI-GSRx, which is a popular open-source GNSS software receiver implemented in MATLAB and based on single-antenna architecture [21]. MATLAB was used to perform initial feature extraction for the PRN with the highest power at a sampling rate of 50 Hz (a time window of 20 ms), as discussed in Section II-B with further training and analysis were conducted in Python v3.9. For visualization, the extracted features for three datasets are shown in Fig. 4. Looking at the Pr feature for DS-3 we can clearly see when and with how much power advantage the spoofed signal appears. Spoofers’ presence can also be seen in C/N_0 values as well, albeit to a small degree. The SQM metrics report no change until the correlators’ peak pull-off begins and goes back to stable values when the takeover is complete. This distortion is evident in features extracted from DS-3 and DS-6. Meanwhile, as the attacker in DS-7 is able to fully align the spoofed signal with the genuine one, the SQM distortions during the peak pull-off are minimal, with only the m_{ratio} and m_{fred} showing some fluctuations. The datasets

TABLE I
TEXAS SPOOFING TEST BATTERY (TEXBAT) SCENARIO SUMMARY

Abrv.	Scenario Info	Spoofing Type	Mobility	Power Adv. (dB)	Synchronization	Duration (sec)	Onset (sec)
-	0: Clean-Static	N/A	Static	N/A	N/A	50-300	N/A
-	1: Clean-Dynamic	N/A	Dynamic	N/A	N/A	50-300	N/A
DS-2	2: Static OverPowered	Time Push	Static	10	Code Phase Prop.	50-300	110
DS-3	3: Static Matched-Power	Time Push	Static	1.3	Frequency Lock Mode	50-300	120
DS-4	4: Static Matched-Power	Position Push	Static	0.4	Frequency Lock Mode	100-350	114
DS-5	5: Dynamic OverPowered	Time Push	Dynamic	9.9	Code Phase Prop.	50-300	102
DS-6	6: Dynamic Matched-Power	Position Push	Dynamic	0.8	Frequency Lock Mode	50-350	105
DS-7	7: Static Matched-Power	Time Push	Static	Matched	Carrier Phase Aligned	100-450	110

were split using a 75-25% split, and to facilitate efficient learning, the features from the training dataset were scaled to a range of [0, 1]. The same scaling was then applied to the test dataset. Overall, we had a total of 107,500 samples to work with, with 35% normal and 65% spoofed samples.

All deep learning (DL) based models like ANN, TS-ANN, and VAE were implemented using the Pytorch [24] Python library. The optimization of the ANN and TS-ANN models was carried out using the Adam optimizer with a learning rate of $1e^{-3}$, batch size of 256, and a training period of 20 epochs. The Binary cross-entropy loss function was used for the optimization process. For the VAE model used in zero-day detection, the Adam optimizer was employed, with a learning rate of $5e^{-4}$ and a batch size of 256 for 50 epochs. The model was trained by minimizing the loss function defined in (10). To evaluate the performance of the ML models, we employed various metrics, including both classification-based measures such as *accuracy*, *precision*, *recall*, and *F1-score*, as well as probability-based metrics such as *probability of detection (PD)*, *false alarm (PFA)*, and *miss-detection (PM)*.

C. Supervised Classification Model Performance

As the first test, we perform standard binary classification analysis. All datasets are concatenated together to generate a composite dataset, which is then split into training and testing subsets. We repeated the training process 10 times (each time with fresh splits) and computed the average results across the 10 repetitions, which are presented in Table II. The table highlights that all models achieved high accuracy, ranging from 98.64% to 99.55%, where K-NN model achieved the top score with RFC being the close second. Similarly, in terms of PD, K-NN and TS-ANN models have achieved the highest scores of 99.61% and 99.57%, respectively. RFC was able to give the lowest false alarms whereas K-NN had the lowest miss rates. Overall, RFC, K-NN, and the TS-ANN models performed best in all metrics considered.

In addition to the six ML methods trained on the proposed feature set, we include three additional models in the comparison as well. These are the ANN based models reported in [12], [14], and Nu-SVM based model reported in [13]. These models were trained using the features outlined in their respective papers. For [12], [14], we used the same ANN architecture as used by the ANN model trained on our feature set. Among them, the best performance metrics belonged to [14]’s ANN-2, slightly lower than our ANN’s scores.

TABLE II
MODEL CLASSIFICATION PERFORMANCE ON THE TEST SET.

Methods	Accuracy	F1	Precision	Recall/PD	PFA	PM
RFC	99.5	99.62	99.73	99.5	0.49	0.5
XGB	98.64	98.95	98.92	98.97	1.97	1.03
K-NN	99.55	99.65	99.69	99.61	0.56	0.39
SVM	98.68	98.97	99.43	98.52	1.03	1.47
ANN	98.79	99.05	99.30	98.81	1.25	1.19
TS-ANN	99.28	99.44	99.31	99.57	1.24	0.43
ANN-2 [14]	98.42	98.55	98.59	98.51	1.74	1.49
Nu-SVM [13]	91.57	93.14	98.01	88.73	3.27	11.27
ANN-3 [12]	92.07	93.63	97.08	90.4	4.91	9.59

Table II shows that the proposed features were informative, resulting in high performing models. Additionally, when we compare the scores obtained from ANN and TS-ANN, it becomes evident that the latter exhibited slightly superior performance across all performance metrics. This observation suggests that the trained $f_1(\cdot)$ model successfully extracted additional information from the multi-correlator fingers, leading to the overall improvement. However, due to the high-frequency nature of the input data, there is a possibility of overfitting in these models. This may be due to the models memorizing the training data rather than learning generalizable patterns, leading to possible performance reduction on new data. To investigate, we analyze the model’s performance under leave-one-out classification settings, discussed next.

D. Leave-One-Out Classification

In this section, we assess the generalization abilities of ML models using the leave-one-out strategy, where one dataset is reserved for testing, and the rest are used for training. This allows us to evaluate how well the models generalize to completely unseen data from various GPS attack scenarios, avoiding memorization of the training data. We selected the RFC, ANN, and TS-ANN models based on their performance and generalization potential. Results from all datasets in Table I are averaged over 10 runs and reported in Table III.

Table III highlights that all three models achieve excellent performance for DS-2 to DS-6 datasets, with detection probabilities close to 100%. Notably, DS-2 and DS-5 share similar characteristics, such as a high relative power advantage and a spoofed signal aligned with the authentic signal’s code phase. Likewise, DS-3, DS-4, and DS-6 are comparable attacks, all having a low relative power advantage and spoofed signals operating in frequency-locked mode. Considering these simi-

TABLE III
PERFORMANCE UNDER LEAVE-ONE-OUT CLASSIFICATION STRATEGY.

Test DS	Random Forest (RFC)			ANN			Two-Stage ANN (TS-ANN)		
	ACC	PD	PFA	ACC	PD	PFA	ACC	PD	PFA
DS-2	98.76	99.93	4.97	97.36	100.00	11.08	99.86	99.93	0.37
DS-3	99.89	100.00	0.40	99.89	100.00	0.40	99.89	100.00	0.40
DS-4	100.00	100.00	0.00	100.00	100.00	0.00	99.18	99.14	0.00
DS-5	98.25	99.99	8.41	98.19	100.00	8.72	99.54	100.00	2.20
DS-6	99.23	99.98	4.18	98.25	100.00	9.70	99.44	100.00	3.11
DS-7	36.71	33.96	0.00	46.44	44.11	0.00	52.73	50.68	0.00
Average	88.80	88.97	2.99	90.02	90.69	4.98	91.77	91.62	1.01

larities, it appears that the presence of DS-3 and DS-4 in the training set could be enough to effectively classify the unseen DS-6 during the test stage, even if DS-6 is not included in the training set.

However, the DS-7 attacking dataset is different from the others due to having matched power with the clean GPS signal and full carrier phase alignment, making it a more subtle attack. As a result, all three models performed poorly when tested against DS-7, with TS-ANN providing the highest detection probability of 50.68%. This 6% increase in PD compared to the ANN's score not only demonstrates effectiveness of the learned SQM metric by the $f_1(\cdot)$ model, but also underscores the significance of the marginal increase in computational load, as it translates directly into performance gains.

Similar to Section IV-C, we also tested models from [12]–[14] under this scenario. For DS-2 to DS-6 datasets, the ANN-2 model trained on [14]'s feature set scored the highest average PD of 99% as compared to [12] and [13], however PD for all three dropped below 40% when tested on DS-7 dataset. This outcome highlights a key challenge associated with supervised learning, whereby the trained models perform worse when tested against significantly different datasets than those used during the training stage.

E. Zero-Day Detector

Unsupervised representation learning methods can be used to address the issue highlighted in the previous subsection. Here, the model learns to identify one class from the training dataset and can be used to classify test samples without being explicitly trained on them. This approach can improve the accuracy of the model on new and unseen datasets. To this end, we only use the authentic datasets, i.e., *clean-static* and *clean-dynamic* datasets to train the proposed VAE model as described in Section III-B1 to be used as the zero-day detector. Once trained, the reconstruction error η of the test sample is used to classify it as clean or spoofed. The threshold τ is computed from the training data using false positive rates (FPR) of 5%, 1%, and 0.1%. The testing scores against all TEXBAT datasets are shown in Table IV.

It is apparent from Table IV that the proposed detector, which has not seen any of the attack datasets during its training, was able to detect them with excellent performance. For attack datasets DS-2 - DS-6, its detection performance is similar to the supervised ML algorithms as seen in Table III. However, for the subtle attack dataset DS-7, it was able to get a detection rate of 86.75% for a low input FPR of

TABLE IV
ZERO-DAY DETECTOR: GPS SPOOFING ATTACK DETECTION UNDER MULTIPLE INPUT FALSE POSITIVE RATES.

FPR/PFA	5%			1%			0.10%		
	ACC	PD	PFA	ACC	PD	PFA	ACC	PD	PFA
Static	97.33	N/A	2.67	99.70	N/A	0.30	99.99	N/A	0.01
Dynamic	92.45	N/A	7.55	98.10	N/A	1.85	99.78	N/A	0.22
DS-2	97.90	99.99	8.76	99.70	99.90	1.01	99.92	99.93	0.10
DS-3	97.72	100.00	8.16	99.60	100.00	1.44	99.87	100.00	0.46
DS-4	99.82	100.00	3.39	100.00	100.00	0.00	100.00	100.00	0.00
DS-5	92.05	100.00	38.29	97.70	100.00	11.10	99.28	100.00	3.47
DS-6	93.97	100.00	33.47	97.40	100.00	14.60	98.88	100.00	6.22
DS-7	92.49	92.50	7.68	89.70	89.30	2.61	87.26	86.75	0.96

0.1% as compared to the 50.62% achieved by the TS-ANN. If we increase the allowed input FPR to 5%, the detection rate further improves to 92.49%. This test clearly signifies the usefulness of representation based learners for GPS spoof detection as we only need to train our models using high quality clean and genuine GPS signals and the resulting model will be robust enough to identify sophisticated attacks as well.

F. Comparison with recent works

In this section, we conduct a comparative analysis of the proposed zero-day spoof detector with statistical hypothesis-based spoof detection methods like [3], [4], [9], [11]. The rationale behind this comparison is that these methods also utilize features accessible at the tracking stage of a standard GPS receiver, such as received power, C/N_0 , and multiple correlation finger outputs. For this evaluation, we focus on the DS-7 scenario dataset, as it represents the most challenging scenario among all the TEXBAT datasets.

Wesson *et al.* [3] employ received power, C/N_0 , and the symmetric differences metric (8). On the other hand, Manfredini *et al.* [11] utilize AGC values from the RF front end and the SQM metric (9), where the AGC values are the inverses of the received power used in our work. Sun *et al.* [4] combine ratio and delta metrics, while Zhou *et al.* [9] use a custom SQM metric computed using five correlator pairs. In all these methods, the detection window is set at 5 seconds (100 samples), and the detection thresholds are computed using [5%, 1%, 0.1%] PFA under the statistical distributions specified in their respective papers. For consistency in comparison, for [3] we combine the normal and multipath hypotheses into a single non-spoof hypothesis, and similarly, the spoofing and jamming hypotheses are combined into a single spoofing hypothesis. Furthermore, in the case of [4], the presented results use the SQM composite based on PFA with the weighting factor $\lambda = 0.5$, as it yielded superior performance.

Using the nomenclature adopted in Table IV, we present the performance of these methods in Table V. Upon examining the PD across all input FPRs, [4] exhibits the lowest performance. This outcome is expected since it is a transient detector, relying on conventional SQM metrics that are sensitive only during the correlator peak pull-off; once the pull-off exceeds 1 chip length, their PD rapidly declines to zero. In contrast, [9] demonstrates high sensitivity to correlation peak distortion, detecting the peak pull-off swiftly due to the utilization of multiple equi-distant correlators. Nevertheless, the PD of [9] is approximately 10 points lower than that of the proposed ZDD

across all input FPRs. On the other hand, despite employing power and SQM metrics, [3] and [11] exhibit lackluster performance in the sophisticated attack scenario of DS-7. This can be attributed to the spoofer's minimal power advantage and their achievement of challenging frequency matching with the genuine GPS received signal. Overall, Table V demonstrates that the proposed ZDD not only achieves a higher detection rate across various input FPRs but also yields lower PFAs.

TABLE V
PERFORMANCE COMPARISON ON TEXBAT SCENARIO DS-7.

Input FPR/PFA	5 %			1 %			0.10 %		
Methods	ACC	PD	PFA	ACC	PD	PFA	ACC	PD	PFA
Wesson <i>et al.</i> [3]	71.25	73.96	8.56	70.76	69.4	5.08	70.18	67.73	1.98
Manfredini <i>et al.</i> [11]	69.35	68.34	11.68	63.22	65.87	7.67	59.68	62.18	3.58
Sun <i>et al.</i> [4]	56.34	57.68	8.64	53.84	54.08	5.68	51.35	52.54	1.28
Zhou <i>et al.</i> [9]	78.55	81.65	9.32	79.89	79.35	6.58	77.76	75.97	2.85
Proposed ZDD	92.49	92.50	7.68	89.70	89.30	2.61	87.26	86.75	0.96

V. CONCLUSION

In this study, we introduced a novel feature set for training ML models to detect GNSS spoofing attacks. The proposed feature set includes power and SQM metrics, which can be extracted in real-time from any off-the-shelf GNSS receiver. Additionally, we proposed a TS-ANN model that learns a new SQM metric from multi-correlator finger values, enhancing the spoofing detection performance. Our analysis of nine ML-based methods, including TS-ANN, using the publicly available TEXBAT dataset, demonstrated high accuracy, correctly detecting over 98.5% of spoofed samples under supervised training. However, under the leave-one-out training strategy, all ML methods struggled to detect the subtle attack scenario DS-7, with TS-ANN achieving the highest PD of only 50.68%.

To address this limitation, we proposed an unsupervised representation learning-based ZDD using a VAE model trained solely on authentic datasets. Our experimental results revealed that the proposed ZDD achieved comparable or even superior performance compared to ML-based methods, particularly excelling in the case of the DS-7 attack scenario, reaching an impressive PD of up to 92.5%. Furthermore, the ZDD outperformed the statistical hypothesis-based methods for the same scenario as well. Overall, our findings highlight the significance of incorporating representation learning-based approaches, in conjunction with traditional supervised learning methods to enhance the resilience of GNSS spoof detection systems against previously unseen attacks.

REFERENCES

- [1] G. Directorate, "Systems engineering and integration interface specification is-gps-200g," 2012.
- [2] T. E. Humphreys, B. M. Ledvina, M. L. Psiaki, B. W. O'Hanlon, P. M. Kintner, *et al.*, "Assessing the spoofing threat: Development of a portable gps civilian spoofer," in *Proceedings of the 21st International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS 2008)*, pp. 2314–2325, 2008.
- [3] K. D. Wesson, J. N. Gross, T. E. Humphreys, and B. L. Evans, "Gnss signal authentication via power and distortion monitoring," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 54, no. 2, pp. 739–754, 2017.
- [4] C. Sun, J. W. Cheong, A. G. Dempster, H. Zhao, and W. Feng, "Gnss spoofing detection by means of signal quality monitoring (sqm) metric combinations," *IEEE Access*, vol. 6, pp. 66428–66441, 2018.

- [5] T. E. Humphreys, J. A. Bhatti, D. Shepard, and K. Wesson, "The texas spoofing test battery: Toward a standard for evaluating gps signal authentication techniques," in *Radionavigation Laboratory Conference Proceedings*, 2012.
- [6] X. Wei, M. Aman, and B. Sikdar, "Exploiting correlation among gps signals to detect gps spoofing in power grids," *IEEE Transactions on Industry Applications*, vol. PP, 2021.
- [7] D. M. Akos, "Who's afraid of the spoofer? gps/gnss spoofing detection via automatic gain control (agc)," *NAVIGATION: Journal of the Institute of Navigation*, vol. 59, no. 4, pp. 281–290, 2012.
- [8] A. M. Khan, N. Iqbal, A. A. Khan, M. F. Khan, and A. Ahmad, "Detection of intermediate spoofing attack on global navigation satellite system receiver through slope based metrics," *The Journal of Navigation*, vol. 73, no. 5, pp. 1052–1068, 2020.
- [9] W. Zhou, Z. Lv, X. Deng, and Y. Ke, "A new induced gnss spoofing detection method based on weighted second-order central moment," *IEEE Sensors Journal*, vol. 22, no. 12, pp. 12064–12078, 2022.
- [10] E. Schmidt, N. Gatsis, and D. Akopian, "A gps spoofing detection and classification correlator-based technique using the lasso," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 56, no. 6, pp. 4224–4237, 2020.
- [11] E. G. Manfredini, D. M. Akos, Y.-H. Chen, S. Lo, T. Walter, and P. Enge, "Effective gps spoofing detection utilizing metrics from commercial receivers," in *Proceedings of the 2018 International Technical Meeting of The Institute of Navigation*, pp. 672–689, 2018.
- [12] S. C. Bose, "Gps spoofing detection by neural network machine learning," *IEEE Aerospace and Electronic Systems Magazine*, vol. 37, no. 6, pp. 18–31, 2022.
- [13] G. Aissou, S. Benouadah, H. E. Alami, and N. Kaabouch, "Instance-based supervised machine learning models for detecting gps spoofing attacks on uas," in *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 0208–0214, 2022.
- [14] E. Shafiee, M. R. Mosavi, and M. Moazedi, "Detection of spoofing attack using machine learning based on multi-layer neural network in single-frequency gps receivers," *The Journal of Navigation*, vol. 71, no. 1, pp. 169–188, 2018.
- [15] M. R. Manesh, J. Kenney, W. C. Hu, V. K. Devabhaktuni, and N. Kaabouch, "Detection of gps spoofing attacks on unmanned aerial systems," in *2019 16th IEEE Annual Consumer Communications & Networking Conference (CCNC)*, pp. 1–6, 2019.
- [16] P. Borhani-Darian, H. Li, P. Wu, and P. Closas, "Deep neural network approach to detect gnss spoofing attacks," in *Proceedings of the 33rd International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS+ 2020)*, pp. 3241–3252, 2020.
- [17] J. Li, X. Zhu, M. Ouyang, W. Li, Z. Chen, and Q. Fu, "Gnss spoofing jamming detection based on generative adversarial network," *IEEE Sensors Journal*, vol. 21, no. 20, pp. 22823–22832, 2021.
- [18] C. Kim, S.-Y. Chang, D. Lee, J. Kim, K. Park, and J. Kim, "Reliable detection of location spoofing and variation attacks," *IEEE Access*, vol. 11, pp. 10813–10825, 2023.
- [19] R. E. Phelts, *Multicorrelator techniques for robust mitigation of threats to GPS signal quality*. Stanford University, 2001.
- [20] O. M. Mubarak and A. G. Dempster, "Performance comparison of elp and delp for multipath detection," in *Proceedings of the 22nd International Technical Meeting of the Satellite Division of The Institute of Navigation (ION GNSS 2009)*, pp. 2276–2283, 2009.
- [21] K. Borre, I. Fernández-Hernández, J. A. López-Salcedo, and M. Z. H. Bhuiyan, *GNSS Software Receivers*. Cambridge University Press, 2022.
- [22] S. Theodoridis, *Machine learning: a Bayesian and optimization perspective*. Academic press, 2015.
- [23] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, pp. 8024–8035, Curran Associates, Inc., 2019.
- [25] T. Zoppi, A. Ceccarelli, and A. Bondavalli, "Unsupervised algorithms to detect zero-day attacks: Strategy and application," *IEEE Access*, vol. 9, pp. 90603–90615, 2021.
- [26] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013.

- [27] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.
- [28] Y. Hua, J. Guo, and H. Zhao, "Deep belief networks and deep learning," in *Proceedings of 2015 International Conference on Intelligent Computing and Internet of Things*, pp. 1–4, IEEE, 2015.
- [29] A. Asperti, D. Evangelista, and E. Loli Piccolomini, "A survey on variational autoencoders from a green ai perspective," *SN Computer Science*, vol. 2, no. 4, pp. 1–23, 2021.



Asif Iqbal received the BS degree in Telecommunication Engineering from NUCES-FAST, Peshawar, Pakistan, MS degree in Wireless Communications from LTH, Lunds University, Sweden, and Ph.D in Electrical & Electronics Engineering from The University of Melbourne, Melbourne, Australia in 2008, 2011, and 2019 respectively.

He is currently working as a Research Fellow with the Department of Electrical & Computer Engineering at the National University of Singapore, Singapore. Dr. Iqbal previously served on the faculty of National University of Computer and Emerging Sciences Pakistan as an Assistant Professor. His research interests include signal processing, deep learning, sparse signal representations, and privacy preserving machine learning.



Muhammad Naveed Aman (S'12-M'17-SM'23) is an Assistant Professor in the University of Nebraska-Lincoln. He received the B.Sc. degree in Computer Systems Engineering from KPK UET, Peshawar, Pakistan, M.Sc. degree in Computer Engineering from the Center for Advanced Studies in Engineering, Islamabad, Pakistan, M.Engg. degree in Industrial and Management Engineering and Ph.D. in Electrical Engineering from the Rensselaer Polytechnic Institute, Troy, NY, USA in 2006, 2008, and 2012, respectively. His research interests include IoT

and network security, hardware systems security and privacy, wireless and mobile networks and stochastic modelling.



Biplab Sikdar (S'98-M'02-SM'09) received the B.Tech. degree in electronics and communication engineering from North Eastern Hill University, Shillong, India, in 1996, the M.Tech. degree in electrical engineering from the Indian Institute of Technology, Kanpur, India, in 1998, and the Ph.D. degree in electrical engineering from the Rensselaer Polytechnic Institute, Troy, NY, USA, in 2001. He was on the faculty of Rensselaer Polytechnic Institute from 2001 to 2013, first as an Assistant and then as an Associate Professor. He is currently

a Professor with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore. His research interests include wireless network, and security for IoT and cyber physical systems.